1. DeepSeek via API

02/22/2025
10X Hub – Derek Yu

Overview

We will likely have to set up a server on which a model is locally run. As such, integrating AI into a website will require API calls to the server at the very least. For testing purposes, we will purely use DeepSeek V3. Here, we will create a basic web application that makes API calls to DeepSeek.

Note: UI/frontend design is not a priority, and won't be perfect. This is just a prototype to see how things would work together.

Note: As of Feb 2025, DeepSeek has limited API access due to network traffic. As such, this project uses OpenRouter's free API to access DeepSeek.

Stack

- Frontend: JS, HTML, CSS
- Backend: Express.js, Node.js

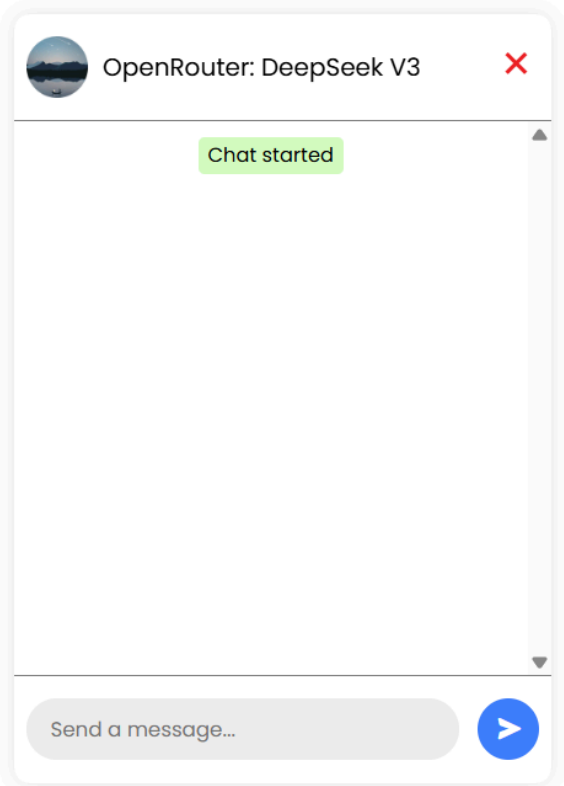
Context

1.1 Chatbot Interface

We will set up an interface where user-AI interactions occur. This follows the conventional "customer service agent" model widely used on modern sites.

This document won't cover frontend & UI design, as this is more likely than not subject to change depending on how the site is structured.

Source Code: [GitHub]



Frontend

2.1 Submit user input

As seen in the source code, the user's input is contained within an input element identified by class "user-input".

There is also a submit button identified by class "send-btn".

The form should submit if the user either presses ENTER while typing in the input box, or clicks on the submit button.

We will create event listeners in index.js. On submit, sendMessage() is run.

```
// on submit, run sendMessage()
document.querySelector(".send-btn").addEventListener("click", ()=>{
    console.log('sending message');
    sendMessage();
});

// on Enter, run sendMessage()
document.querySelector(".user-input").addEventListener("keydown",
function(e) {
    if (e.keyCode == 13 && !e.shiftKey && sendBtn.disabled==false) {
        e.preventDefault();
        sendMessage();
    }
});
```

2.2 Processing user input

The application frontend somehow needs to send the user's input over to the server (which we will be simulating with Express), which is then used to make an API call.

To do this, we'll create a function sendMessage(). When called, sendMessage() uses the fetch API to make a POST request, which sends the user's input to the server (backend) to make an API call. Then, it processes the response received from the backend after the API call has been completed.

sendMessage()

Source code: GitHub

1. Takes user input
2. Reset input box
3. Disable submit
4. Create user chat bubble with input
5. POST user input to backend, fetch response as 'data'
6. Check data for errors. If none, create an AI chat bubble containing the model's response. Else, handle errors accordingly.
7. Re-enable submit

Backend

3.1 Server

For this project, we will use Express to host a server on port 5000. Note that this server is local and works locally on the device only. We'll likely opt for running a server on the cloud with AWS, Google Cloud, or a similar cloud platform (on which a custom model may be run) in the future.

Just like any other website, we'll create "routes" within our server, each route containing different code and protocols. This makes our server much more modular and scalable. In this case, we have different routes for different models and APIs we want to access:
Creating routes

```
app.use("/openai", openai)
app.use("/deepseek", deepseek)
app.use("/openrouter", openrouter)
```

localhost:5000/deepseek/v3 → DeepSeek API: V3 (source code: GitHub)
localhost:5000/openai/4o → OpenAI API: GPT-4o (source code: GitHub)
localhost:5000/openrouter/deepseek/v3 → OpenRouter API: DeepSeek V3 (code: GitHub)

For this project, we will use OpenRouter to access DeepSeek V3 as DeepSeek has blocked API access as of Feb 2025.

3.2 API Keys

To make API calls, you'll need an API key. Each API will have its own API keys, which can be generated on their respective websites. However, these keys are PRIVATE, and should NEVER be publicly shared. You will not find any API keys in the source code.

For this project, API keys are stored as "environment variables" in a file ".env" in the 'backend' folder. You will need to create your own .env file and store your own API keys to access the APIs.

OpenAI: https://platform.openai.com/api-keys
DeepSeek: https://platform.deepseek.com/api_keys
OpenRouter: https://openrouter.ai/settings/keys

Here is an example of what your .env file should look like: GitHub
backend/.env

```
DEEPSEEK_1=ENTER_KEY_HERE
OPEN_ROUTER_1=ENTER_KEY_HERE
OPENAI_1=ENTER_KEY_HERE
```

3.3 DeepSeek API Call (OpenRouter)

For this project, we will use OpenRouter's API to indirectly access DeepSeek.

We need our backend to take the user input (passed in with POST) as a prompt and make API calls with the selected model. To do this, we could make another POST request from the backend to the model's API. However, for this project, we'll use OpenAI's built-in functions (which I think are easier to work with), since we are already using Node.js anyway.

Let sendMessage() → POST [user input] to localhost:5000/openrouter/deepseek/v3

Whenever sendMessage() initiates a POST request to the server, the corresponding server route picks up the POST request. In this case, openRouter.js. The server will then use the prompt to create an API call to DeepSeek and return the AI's response as a response to the POST's sender - sendMessage().

If POST (user input) to localhost:5000/openrouter/deepseek/v3, then:

1. Grab user input from POST request as "prompt"
2. Create API call: Provide model, AI behavioral prompt, User prompt.
3. Store AI's response received from API call as "response"
4. Check response for errors, then return response to sender.

3.4 Model Behavior

Model behavior is an important component in any API call. We can provide "context" to a model as a prompt, in the form "You are …", to predefine how we would like the model to respond to the user's prompt - role, tone, personality, etc.

Using this, we can also provide the model with additional information about the context. In our case, the model serves as a customer service assistant. As such, it may be useful to provide a collection of FAQs with answers to serve as examples. For this project, we will additionally provide a JSON file with random data for testing purposes.

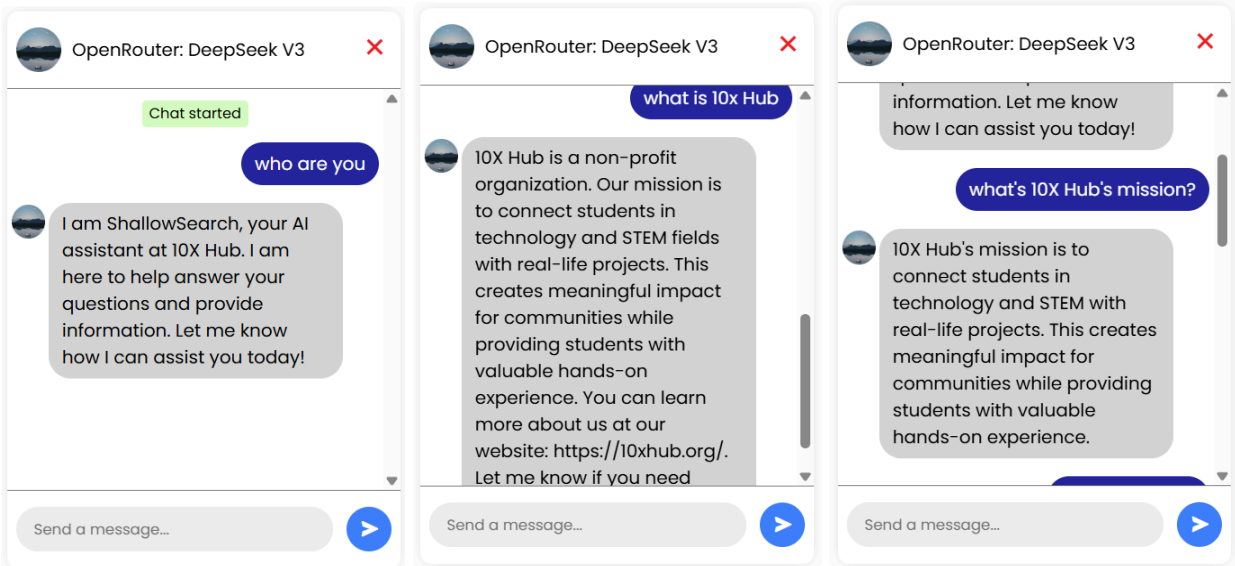
```
{
role: "system",
content: `You are Megatron, 10X Hub's AI assistant. Here is some data
about some of us at 10X Hub that may help answer some of the user's
questions: ${readFile('./users.json')}. ${info}`
}
```

In users.json, there is a collection of dummy employees.
In info.js, there is a collection of FAQ's and response instructions.

Demo

Here are a few sample outputs from the chatbot. While it does answer questions, it does so in a very generic manner, as it does not have access to all the information it needs.

HTML (download & open file) - GitHub
Video (YouTube) - https://youtu.be/Mp6cJg9IQ80



Next Steps

Through this project, we demonstrated how AI can be leveraged in web application chatbots. Although it's not perfect (as expected), this serves as a starting point as to what we are trying to build.

Many of the responses given by the chatbot feel too "bulky". This is because we are using nothing but a Large Language Model to communicate with the user. We've "faked" training the model by passing in specific behavioral information along with each prompt provided by the user. However, this isn't how a customer service assistant should feel.

Most customer service bots tend to give short answers and have limited response capabilities - usually providing users with resources related to their inquiry. Most of the ones I've worked with only respond with preset responses - instead of processing the user's input and responding with an LLM, these models seemingly use vector embeddings to pinpoint the most relevant pre-existing answer to provide.

While we could take the natural language processing route, there may be more work to be done in terms of fine-tuning a model that works perfectly as an AI assistant for the site. This means potentially training a custom model, and likely running it on a cloud server. And I have no clue how to do any of that yet which is great.

Next step - run a model locally with Ollama. For the development phase, I can think of two options:

1. set up a remote SSH connection with my PC to act as the "server"
2. Set up a remote SSH connection with an AWS cloud instance (if less than $300/month)

I will attempt to run and configure a custom DeepSeek model. Following that, we will have to consider the "chatbot" requirements to determine the model's behavior.